ROGUE BATTLEBOND

Rapporto di Analisi Tecnica e Architettura Software

Versione: 1.0
Tipo: Web SPA (Single-File)
Ambiente: Client-Side

0	< 100 KB	< 100 ms	Vanilla JS
DIPENDENZE ESTERNE	FOOTPRINT MEMORIA	TEMPO CARICAMENTO	MOTORE TECNOLOGICO

1. ARCHITETTURA GENERALE DEL SISTEMA

Il gioco è strutturato come una **Single-File Web Application (SPA)** monolitica realizzata in HTML5, CSS3 e JavaScript puro. L'intero ciclo di vita dell'applicazione risiede sul client senza necessità di un backend dedicato per il loop di gioco.

Paradigma Client-Side Pure

Esecuzione diretta all'interno dell'engine JS del browser. Zero build-step richiesto (nessuna fase di compilazione o bundling tramite Webpack/Vite).

Pattern di Gestione Stato

Stato monolitico centralizzato in memoria per la gestione unidirezionale dei dati: statistiche giocatore, progressi mappa e party attivo.

2. COMPONENTI LOGICI & MODULI DI GIOCO

A. Game State Manager

Mantiene le variabili globali del giocatore: salute (HP), risorse/monete accumulati, piano corrente (Floor) e abilità con relativi contatori di cooldown.

B. Turn-Based Battle Engine

Gestisce il combattimento deterministico e stocastico: calcolo attacco/difesa, modificatori elementali, gestione dei cooldown e IA nemica basata su regole.

C. Generatore Mappa Roguelike

Algoritmo procedurale basato su nodi decisionali (Combattimento, Shop, Evento, Riposo, Boss) con allocazione dinamica di ricompense e incontri.

D. Interfaccia & Rendering Layer

Aggiornamento reattivo del DOM tramite CSS Flexbox/Grid e transizioni animate per il feedback di danno subito, selezione abilità e stato del turno.

3. PUNTI DI FORZA TECNOLOGICI

Efficienza Primaria

Assenza totale di overhead di framework. Prestazioni di esecuzione massime e consumi minimi di CPU/RAM.

Portabilità Totale

Distribuibile istantaneamente come file statico, GitHub Pages o incapsulabile in Electron/Capacitor per Desktop e Mobile.

Prototipazione

Modifica rapida e ciclo di test immediato senza necessità di fasi di rebuild o ricompilazione dei moduli.

4. DIAGNOSI CRITICITÀ & STRATEGIE DI REFACTORING

Area Critica	Problema Rilevato	Soluzione Architetturale
Accoppiamento (Coupling)	La logica di calcolo del danno è direttamente legata alla manipolazione degli elementi HTML.	Integrazione di un sistema ad eventi (Event Bus / Pub-Sub) per disaccoppiare Engine e UI.
Scalabilità Dati	Definizione inline dei dati di Bond, Nemici e Skill direttamente nel codice sorgente JS.	Estrapolazione dei bilanciamenti in file di configurazione esterni `.json`.
Persistenza Stato	Assenza di salvataggio dello stato di gioco in caso di refresh della pagina o chiusura.	Serializzazione dello stato nel `localStorage` ad ogni completamento di nodo.
Manutenibilità	File unico monolitico difficile da estendere con l'aumentare dei contenuti.	Migrazione a moduli standard ES6 (`import` / `export`) suddivisi per ambito.

5. ROADMAP SUGGERITA PER LO SVILUPPO

Fase	Obiettivo Tecnico	Intervento Chiave	Priorità
Fase 1	Modularizzazione ES6	Separazione del codice monolitico in moduli JS (`engine/`, `ui/`, `data/`).	ALTA
Fase 2	Sistema Save/Load	Integrazione di salvataggi automatici trasparenti tramite `localStorage`.	ALTA
Fase 3	Database JSON	Esternalizzazione di mostri, abilità e tassi di drop in file configurabili.	MEDIA
Fase 4	Polish & Visual FX	Implementazione di animazioni particellari Canvas per colpi critici ed effetti.	BASSA

Sintesi Strategica

Il progetto presenta una solida base prestazionale grazie all'architettura leggera. L'implementazione delle prime due fasi di refactoring (ES6 Modules e Salvataggio LocalStorage) permetterà di passare da un prototipo a un prodotto scalabile e pronto per la distribuzione pubblica.